croit

TECHNICAL ARCHITECTURE

croit-flashstore-osd
An NVMe-only Ceph OSD in Rust

A wire-compatible drop-in for `ceph-osd`: kernel NVMe (SPDK optional), kernel TCP over io_uring with an optional RDMA upgrade, one pinned thread per core, ownership instead of locks. Design per layer with status and ADR, guarantees, operations, method, limits and the changes since 2026-10-28.

As of: 2026-11-11 20:50

Sources: repository docs (doc/adr)

TAKEAWAY **Evidence is from lab runs and bare-metal checks.**

MOTIVATION

Modern NVMe is fast; the classic OSD stack spends its CPU elsewhere

A modern NVMe drive serves very high IOPS at small IO sizes. Per-IO CPU, not device speed, sets the classic stack's limit.

- **Threads and queues:** shared op queues, wake-ups and locks between them.
- **Metadata:** RocksDB (LSM, compaction, WAL) plus BlueFS under every object.
- **Kernel paths:** block layer and TCP/epoll cost syscalls, copies and context switches per IO.
- **Rule (project rules, AGENTS.md):** latency, IOPS, throughput and space at or above classic BlueStore on the same hardware. CPU per op is secondary: higher is fine when IOPS and bandwidth are higher. A loss is a bug to profile and fix.

Charter: maximum IOPS per core and cycles per IO on modern CPUs, NICs and NVMe. No HDD, no BlueStore on-disk compatibility. Migration = add croit-flashstore-osd OSDs, remove old ones.

COST CENTER	CLASSIC CEPH-OSD + BLUESTORE	CROIT-FLASHSTORE-OSD
Execution	thread pools, shared queues, mclock scheduler	one pinned thread per core, busy-polling while work exists, asleep when quiet; PG owned by one core (ADR 0006, 0031, 0032)
Metadata	RocksDB LSM + BlueFS	COW B+tree + redo WAL per shard, no LSM by default; proposed opt-in OMAP engines (ADR 0021, 0610, 0681, 0727)
NVMe	kernel block layer	kernel NVMe through io_uring, one ring per core (ADR 0169; the packaged `prod` profile ships this tier, ADR 0682); optional SPDK polled driver, one qpair per core (prod-full, ADR 0004)
Network	kernel TCP	kernel TCP via io_uring (knet, the only transport), optional RDMA upgrade of its sessions (ADR 0053, 0469, 0682)
Memory	autotuned caches, heap churn	per-core pools, shared payload `Bytes`, one envelope from `osd_memory_target` (ADR 0033, 0062, 0175)
Sync	mutexes between op, commit and messenger threads	ownership; cross-core via SPSC rings, atomic ingress credits and immutable shared `Arc` data (OsdMap)

TAKEAWAY **On the same cores, CPU per IO, not device speed, limits the classic stack.**

Six rules shape every decision, each one recorded as an ADR

1 · KISS

Complex code is never fast. Scope is fixed in ADR 0013: a cluster of only croit-flashstore-osd, replicated size 1-3, RBD incl. metadata and `.mgr`; EC (both roles), RGW cls and pool snaps are in scope since 2026-10-20 (ADR 0013, 0010); format-1 RBD directories use TMAP ops. No HDD, no legacy, no "support everything".

2 · CLEAN ROOM

Ceph source is reference only. It becomes an own-words spec (); code is written from the spec, never copied or translated (ADR 0003). Permissive dependencies only. License: LGPL-3.0-only (client libs MPL-2.0; docs CC-BY-SA-4.0), ADR 0164.

3 · DROP-IN WIRE COMPATIBILITY

Same msgr2, cephx, OSDMap and CRUSH as the pinned Ceph version (main: v21.2.0, branch v20.2.4). A fresh NVMe plus cluster credentials is enough. It identifies as BlueStore (ADR 0019). Mixed acting sets with classic OSDs must work (ADR 0013).

4 · ACK ONLY WHEN DURABLE

Reply only after data, metadata and PG log entry are on media. PLP NVMe is required: a completion is durable, no flush on the hot path. mkfs refuses devices with a volatile write cache; the lab override `mkfs --allow-vwc` uses FUA (ADR 0008).

5 · OWNERSHIP, NOT LOCKS

A PG belongs to exactly one core. Its state is plain single-threaded data. Parallelism comes from many PGs x many cores x one device queue per core (ADR 0006, 0008).

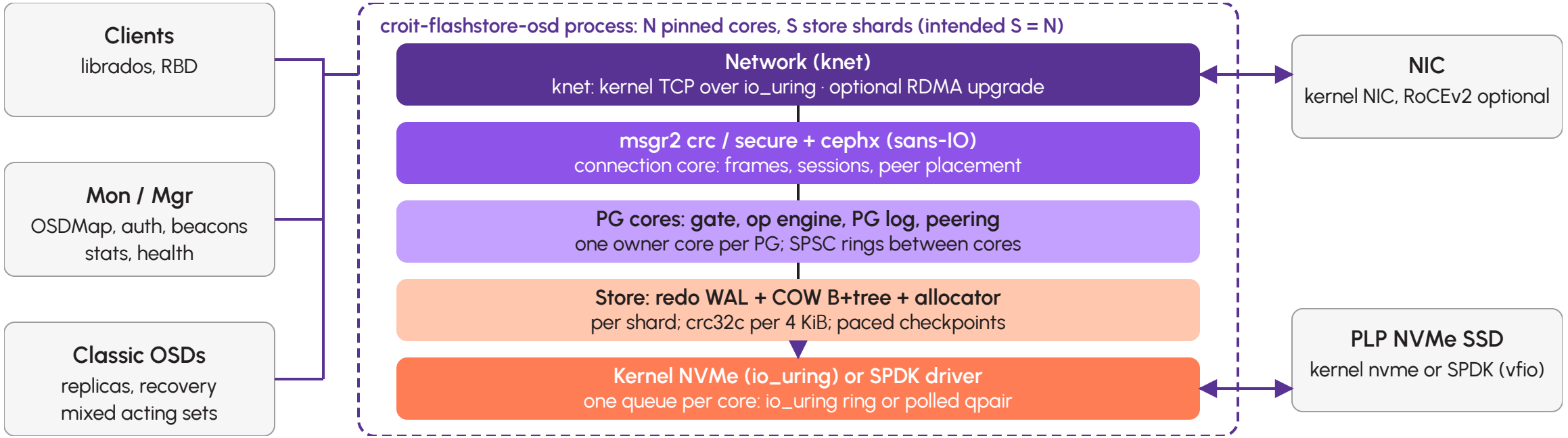
6 · NEVER CONFUSE THE CLUSTER

Classic OSDs abort on malformed or misrouted PG messages. Unsupported pool types are declined with a fixed classic reply set; they are fully passive only when the operator turns the decline switch or EC serving off, or when the OSD holds data of the pool. Every outbound PG message passes one choke point (ADR 0039, 0050).

Lab and dev backends sit behind cargo features with static dispatch, zero cost in production (ADR 0024). Upstream test suites are temporary parity oracles; every feature ships with its own tests.

TAKEAWAY Each rule is an ADR; the prime safety rule outranks speed.

One process per NVMe device, speaking classic Ceph on the outside



How to read: the dashed box is one OS process. Requests enter from the left, flow down the layers and reach the SSD on the right.

INSIDE

Crates under `crates/` (msgr2, net, nvme, rt, crush, ec, ...) and `services/osd/`. Rust throughout: shared crates, services (osd) and tools.

CORE ROLES

Core 0 also runs control: mon/mgr sessions, boot, osdmap, aggregation. Every core, core 0 included, hosts links and PGs (ADR 0031).

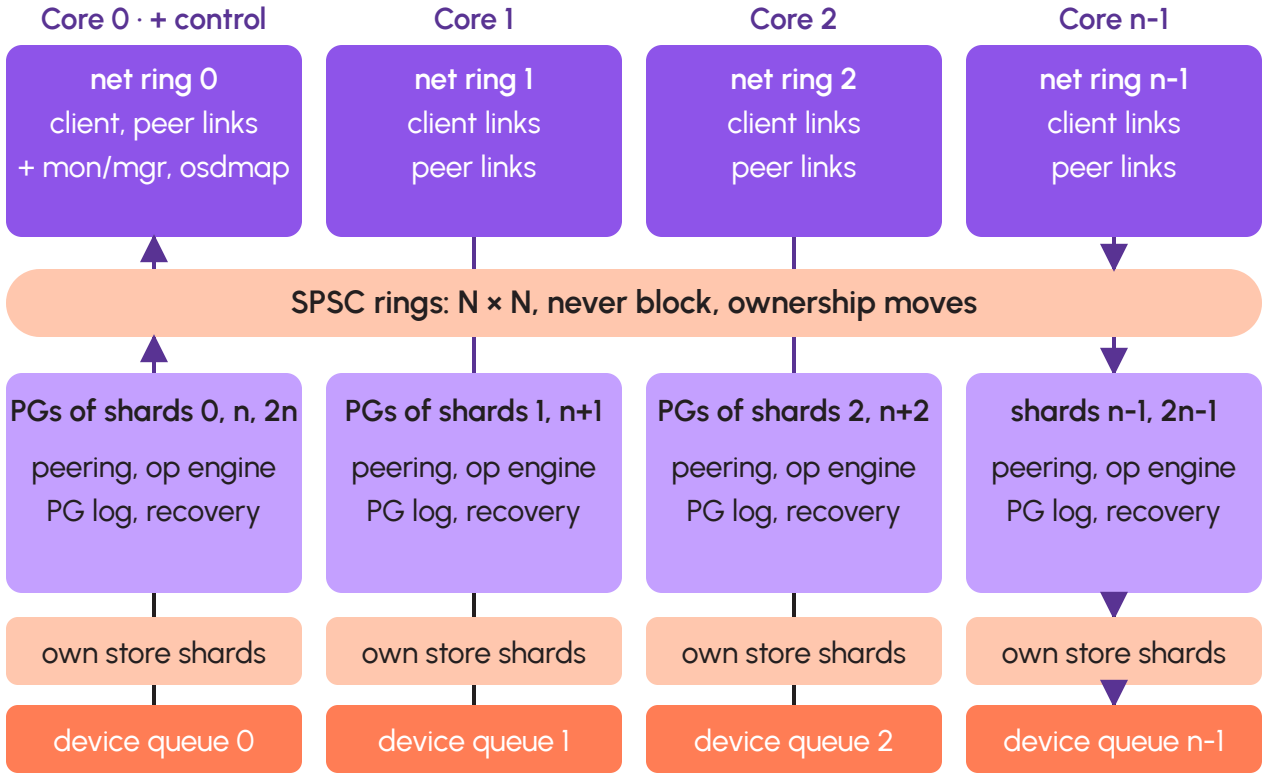
OUTSIDE

Looks like `ceph-osd` to mon, mgr, clients and peers: admin socket, perf counters, beacons, crash metadata.

TAKEAWAY One process per NVMe device; classic peers see an ordinary ceph-osd.

Thread-per-core, PG-per-core: ownership replaces locks

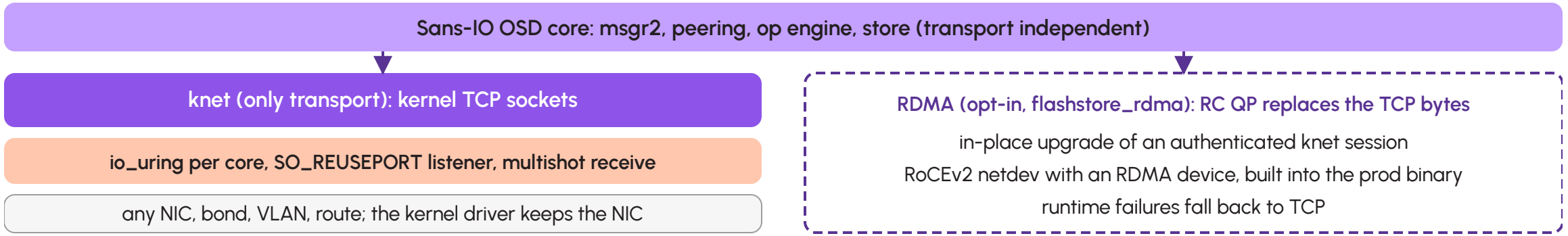
- **One `0sd` per core.** Core 0 also does control; every core hosts links and PGs. Poll loop: network rx, msgr2, op, device submit and complete, tx (ADR 0006, 0031).
- **PG → core = store shard mod cores**. Core 0 places every arriving PG: fewest PGs of its pool, then fewest in total, then lowest core index; split children inherit the parent's shard (ADR 0639). Shards are fixed at mkfs; intended layout: cores = shards (ADR 0160). Core *c* of *n* owns shards *s* mod *n* = *c*; another core count starts with a warning on the kernel tier, SPDK needs a core per shard.
- **Connection core ≠ PG core.** Owned messages move over N×N SPSC rings; ownership travels with the value.
- **Default cores = CPUs in the affinity mask** (cpuset, systemd, taskset), at most the store's shards. Several OSDs per host, each in its own cpuset.
- **L3-aware placement** (ADR 0063): fill one L3 domain first, and keep OSD cores off the CPUs that serve NIC interrupts when enough other CPUs remain (otherwise they share them; self-tuned, see the tuning pages).
- **Idle:** a core with a quiet store and no mesh backlog sleeps in its ring (eventfd wake), so many OSDs per host do not burn every core (ADR 0032, 0053).



How to read: each column is one pinned core, top to bottom: network, PG work, store, device queue (SPDK qpair or io_uring ring). Lilac marks PG work. Between columns: SPSC rings; PG and store state are core-owned, no lock on them.

TAKEAWAY **Parallelism = PGs x cores; PG state never needs a lock.**

One transport, knet (kernel TCP over io_uring), plus an RDMA upgrade



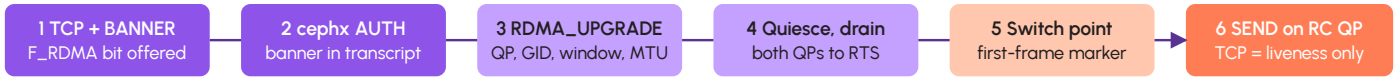
PART	USE WHEN	DESIGN
knet	Always (ADR 0053). Bonds, VLAN trunks, routed hosts, any NIC the kernel drives; no VF budget, no NIC hugepage setup.	Per core: own io_uring, listener, multishot receive into a provided-buffer ring; large sends zero copy; large receives land in the segment buffer. io_uring features are probed at start, never gated by kernel version. Source-port steering keeps a peer on its core (ADR 0054). zcrx receive is probed per queue and opt-in (`--knet-zcrx auto`) until a bare-metal A/B shows a gain (ADR 0464).
rdma	RoCEv2 NIC on the OSD address, both peers opt in (`flashstore_rdma`). Next page.	Not a tier: an upgrade of established knet sessions (ADR 0469, 0470).

How to read: the top bar is the same code for every transport. Purple = transport, peach = driver, grey = what the transport needs from the NIC.

TAKEAWAY knet runs anywhere and is the only transport; RDMA upgrades its sessions.

STATUS knet on, the only transport · RDMA opt-in, off · **OPEN** zcrx rollout

RDMA (RoCEv2): an opt-in in-place upgrade of authenticated knet sessions



- **Same session, same auth:** the upgrade sits after the last AUTH frame, before CLIENT_IDENT. The bit is never in `required_features`; it is inside the signed transcript, so stripping or adding it faults the connection (ADR 0469).
- **Eligible:** only where the peer's owner core is the landing core; handed-off and relayed links and peers on the same host stay TCP. Classic OSDs, mons, mgrs, heartbeats and older croit-flashstore-osd stay TCP. A decline or failure is a TCP connection, never an error.
- **Data path:** one RC QP per connection, one CQ and one SRQ per core, SEND only. Receive credits = posted buffers, granted dynamically up to a fixed window ceiling. Large payloads leave zero copy from the registered hugepage heap.
- **Control calls** (QP create/modify, memory registration) run on a helper thread, never on a data-path core.
- **Liveness:** the idle TCP socket reports a dead peer; a graceful close is fenced by a signaled zero-length SEND. A QP error is a session fault: msgr2 reconnect over TCP. A last-WQE event that arrives before the teardown asks is kept per QP (F1 fix), so a fault never leaks a QP or its credits.

TOPIC	RULE (ADR 0469, 0470)
Enable	`flashstore_rdma = true`, off by default; knet only; built into the production binary (static rdma-core).
Device	RoCEv2 GID of the netdev that carries the msgr2 IP (PF or VF); RoCEv1 unsupported. croit-flashstore-osd adds no IP, VLAN or GID.
Path MTU	Smaller of both ends. Standard 1500-byte networks work; the largest RoCE MTU (jumbo netdev MTU) is recommended and `--check` warns below it. Too small a path MTU stays TCP.
Fabric	Lossy works. Lossless recommended: PFC on one priority, a fixed DSCP (no option), ECN, DCQCN. The host network is never changed.
Memory	Registered send / receive arenas: own budget `flashstore_rdma_memory`, outside `osd_memory_target`.
Visible	`rdma_*` perf counters, `rdma status`, cluster-log warnings; no silent fallback.
Faults	Fault campaign: outcome Ceph-compatible; F1 (QP + credit leak) and F2 (GID index cached) fixed, ADR 0469.

TAKEAWAY **Only the byte carrier changes; a failed upgrade or RDMA error falls back to the TCP path of the same session (IPv4 only).**

STATUS opt-in · F1 / F2 fixed · READ placement (ADR 0471, proposed): off, lab qualification only · **GATE BEFORE DEFAULT-ON:** at least knet in every discipline, RDMA cycles counted, no silent fallback.

msgr2 with few passes over the payload, sessions pinned to a core

- **Sans-IO msgr2 + cephx** (ADR 0015). crc or secure follows `ms_*_mode` at runtime (ADR 0036); the single-stream throughput target uses crc.
- **Receive and store:** one copy into the segment, block crcs fused in (ADR 0061); the store DMAs from that buffer and reuses the crcs. Payload travels as shared `Bytes` (ADR 0033): replica fan-out is a refcount.
- **knet receive:** a big data segment is received directly into its buffer (in pieces, crc per piece); large receives of a co-located stream are spread to helper cores of the mesh (needs more than one core); fixed default threshold.
- **Transmit:** larger data segments leave by reference (ADR 0033); knet sends large batches zero copy, except to a same-host peer.
- **Reads by reference:** a read is a list of pieces (DMA buffers or prefetch runs with their crcs) sent as msgr2 parts, crc checked in place (ADR 0043, 0163); pinned buffers show as `read_shared`.

4 MIB CLIENT WRITE, REPLICATED, PRIMARY: SWEEPS OVER THE PAYLOAD

STEP	COST
NIC DMA → rx ring	DMA
rx → segment buffer, crc per 4 KiB block	1 copy + block crcs, fused
MOSDOp decode, `WriteReq` skeleton	0
store: NVMe DMA from the payload (zero copy)	0, block crcs reused
shard validate (reuses stored crcs)	0
repop data segment (wire bytes unchanged)	0, sent in parts (ADR 0061)
msgr2 tx data-segment crc (crc mode)	0 (`crc32c_combine`)
tx to N peers (crc mode)	zero copy, shared
CPU sweeps over the payload	**1**

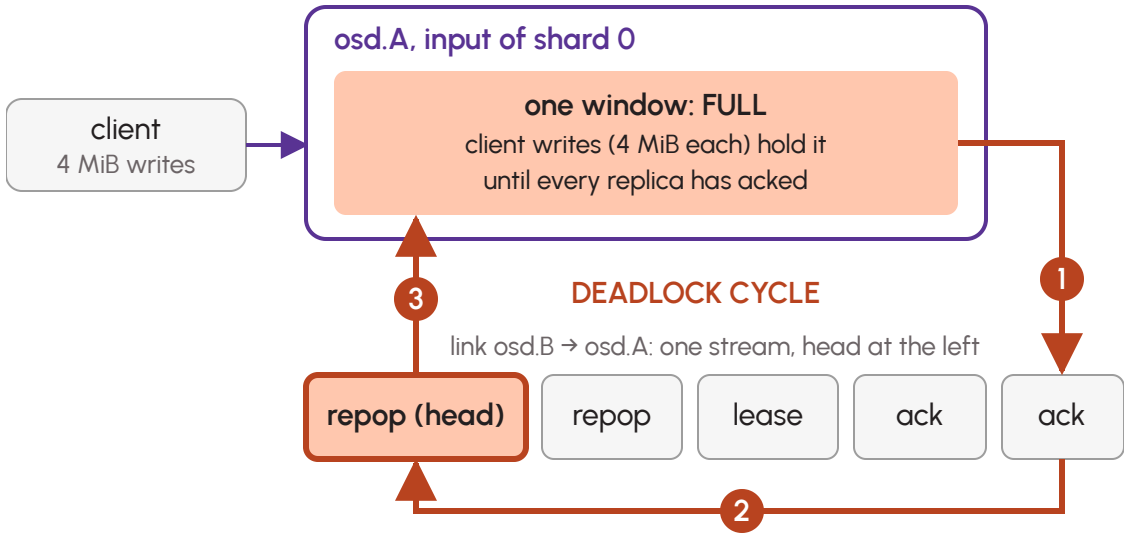
How to read: each row is one CPU pass over the 4 MiB payload; fewer passes means less memory traffic per byte. Replica: the receive buffer carries a pad so the payload sits on the 4 KiB grid (ADR 0061: placement by `data_off`, every frame and size): same single pass; a payload not at the expected offset (classic primary, small segment) takes a staged copy plus a crc pass. Source: code path (ADR 0061); a code-path count, not a measurement.

TAKEAWAY **Primary: 1 sweep over the payload, then the store writes from the receive buffer. Repops go out by reference.**

INPUT BUDGET client and peer data have separate pools, reserved at the frame header (ADR 0084, 0175; next page).

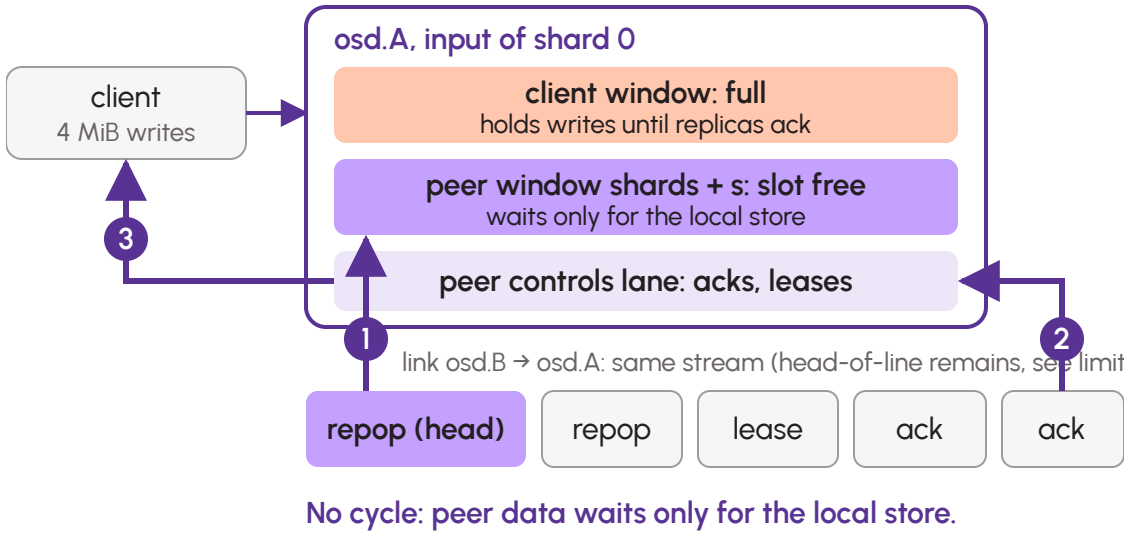
Flow control between OSDs: client and peer input lanes

WHY NOT ONE SHARED WINDOW PER SHARD: IT CAN DEADLOCK



1 writes wait for acks · **2** acks sit behind the head of the same stream · **3** the head repop waits for a window slot. Nothing moves; leases stop too (laggy PGs).

DESIGN: SEPARATE CLIENT AND PEER WINDOWS AND LANES



No cycle: peer data waits only for the local store.

1 peer repop takes its own window slot · **2** acks and leases take the controls lane · **3** the ack completes the client write.

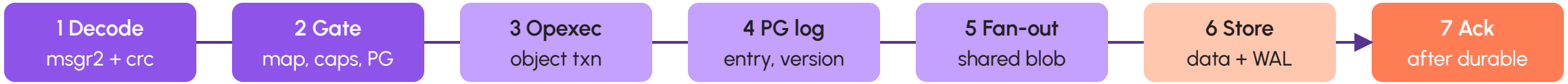
Same bounds per class (bytes and input count per shard); no wire or disk change. Covers the count variant too (many omap writes on one object). Sender-side repop credits: built, **default off**. Byte pools: ingress is a fixed fraction of `osd_memory_target` (with a floor), half for client and half for peer data; a frame reserves at its header (ADR 0175).

TAKEAWAY **Peer traffic gets its own window and lane, so a full client window cannot form a deadlock cycle with the acks it waits for.**

LIMIT one stream still carries all shards: a peer repop waiting for a congested local shard delays later traffic on its link until that store drains (ADR 0084).

OP PATH

A client write: seven stages on two cores, acked after durability



Who runs it:

connection core	SPSC hop	PG core: owner of the PG and of its store shard; fan-out and store run in parallel	hop back, tx

Everything between the hops is single-threaded, lock-free state of one core.

How to read: boxes are the stages in order; the bar below shows which core runs them. Replica nodes run stages 1-2 and 4-6 for the repop and reply to the primary. Next pages: the data movement of replicated and EC paths.

REPLICATED WRITE

Primary appends the PG log entry, builds one shared txn blob, sends MOSDRepOp to each replica and applies locally in parallel. Client ack after every acting member is durable (ADR 0008).

READS BY REFERENCE

A client read returns pieces of the device DMA buffers or prefetch runs, crc checked in place, sent as msgr2 parts (ADR 0043). Layouts with small device pieces take the copy path. Only a cold tree page costs a device read before the data read.

HOT-PATH HYGIENE

Replicated 4k write CPU cut by lazy PG info, a dup index and no per-object omap counters (ADR 0021). Cold tree pages park the op instead of spinning the core.

TAKEAWAY **One hop in, one hop out; the PG core owns everything in between.**

Op QoS: classic scheduler options as per-core admission shares

- **Status:** production, on; ADR 0752 (accepted); A/B vs classic open. No op queue exists (thread per core, run to completion); every core owns one scheduler of three tag triples, a virtual time and counters: no lock, no allocation.
- **Client ops are never held:** they run as they arrive and are charged (class client, priority from the MOSDOp header, cost = data bytes with the mclock floor).
- **Background units ask before they start:** recovery and backfill dispatch, snap trim round, scrub chunk, PG deletion step are admitted or wait. Classes: client, background recovery, background best effort.
- **mclock** (default `osd_op_queue`): reservation, weight, limit per class from `osd_mclock_profile` (`balanced`, `high_client_ops`, `high_recovery_ops`, `custom`); work conserving. A `custom` client limit/reservation is not enforced on client ops. **wpq:** weight = priority, cost-accounted, plus the recovery, snap trim, scrub and delete sleeps.
- **Cut-off:** units at or above `osd_op_queue_cut_off` are strict: never held or charged. Peering events, replica sub-ops and replies, EC sub-ops run inline.
- **Capacity:** no boot benchmark; per-core capacity = `osd_mclock_max_sequential_bandwidth_ssd` / cores, cost floor from `osd_mclock_max_capacity_iops_ssd`. `osd_recovery_max_single_start` gives a PG a burst of object starts.

RUNTIME	RULE (ADR 0752)
Latched at start	`osd_op_queue`, `osd_op_queue_cut_off`
Applies at once	profile, `osd_mclock_scheduler_*`, capacities, priorities, costs; an invalid profile is refused at `config set`
No effect, noted in the log	`osd_op_num_shards*`, `osd_op_num_threads_per_shard*` (one thread per core), `ms_crc_header`, `osd_skip_data_digest`
Visible	per-core counters (`strict_ops`, class charges), ADR 0752

Limit: shares are per core: a core admits against its own capacity share, not an OSD-wide queue. Recovery budgets (`osd_max_backfills`, scrub slots) are OSD-wide, kept by core 0 (ADR 0031, 0083).

TAKEAWAY **Client IO is charged, background work is admitted; the scheduler options behave like classic without an op queue.**

STATUS on · ADR 0752 · amends ADR 0062 point 4 (options no longer ignored).

Read path: per-shard data cache, order kept per object

- **Status:** production, on; ADR 0043, ADR 0633, ADR 0021.
- **Per-shard data cache, default on** (`flashstore_store_pf_retain`). A run that served a read whole moves into a pinless per-shard cache (segmented LRU, shared bytes, block crcs kept); write tails enter it at submit; scrub bypasses it. Size: a floor plus the unused tree page budget; metadata keeps priority (ADR 0633). Tree pages and a point cache sit beside it.
- **Prefetch table first:** after the tree pages are resident the op gate probes the data runs and reads them into a per-shard table (bounded entries and blocks); entries are single use (a run served whole moves into the data cache without a copy); unconsumed ones expire after a short time. A full table or device queue parks the op; only never-prefetched runs (huge ops, some hazards) read synchronously.
- **One pass over the payload:** the read copy is fused with the per-block crc check; the reply crc is combined from the stored block crcs without touching the data; the result buffer is not zero-filled; EC primary reads ask all shards in one pass.
- **Order is per object, not per PG:** ops of one object run in arrival order through the gate (inflight < waiters | parked < gate); pure reads do not wait for each other; reads flagged RWORDERED are ordered with writes. Ops on other objects overtake a cold one.

STEP	WHAT HAPPENS
1 gate	Order per object; probe tree pages when the cache is cold (`Store::warm`).
2 prefetch	`warm_data`: device runs into the shard table; the op parks on a miss.
3 serve	`read_runs_into`: copy + block crc in one pass, buffer by reference when whole blocks.
4 reply	crc from the store (`CrcHint`), payload by reference to the connection core.

Limit: a hot set larger than the cache is re-read from the device; over RDMA a hit is copied into the send arena.

TAKEAWAY **Reads do not block the PG core; re-reads hit the per-shard data cache; ordering is per object.**

STATUS on · ADR 0043 (per-object order, crc at completion).

Replicated write: one receive copy, then payload by reference

#	STEP	CORE	PAYLOAD MOVEMENT	DEVICE
1	Client to primary: rx, msgr2 frame, block crcs	connection core	1 copy into the segment buffer, block crcs fused for large segments (ADR 0061)	—
2	Hop, gate, opexec, PG log entry	PG core	Large payloads stay out of line as the request's `Bytes` (skeleton txn); small ones go into the v9 blob	—
3	Local store: translate, tree apply, submit	PG core	DMA by reference when the buffer is DMA-able, block crcs reused; else one fused copy + crc into a DMA buffer	data + WAL unit, issued together
4	Repop encode, one per replica, sent by the peer link's core	PG core, link core	Plain v9 wire bytes in frame parts, by reference; segment crc joined from part crcs (ADR 0061)	—
5	Replica: rx, relay if the link landed off its owner core, hop, log append, store	replica cores	1 receive copy with the payload on the 4 KiB grid (ADR 0061); relay copy only off-core; DMA by reference	data + WAL unit
6	Replica commit reply to the primary	replica cores	—	after both durable
7	Primary acks the client	PG core, connection core	—	local and all replicas durable, LSN order

DURABILITY

No step moves the ack before every acting member is durable; group commit shares WAL units under load (ADR 0021).

LARGE WRITES

Device commands are limited by the backend's maximum transfer; the store submits data in waves behind the device queue and never refuses a replica transaction (ADR 0066).

REMOVED / BUILT

Fan-out cut-through: removed (ADR 0061). Pre-write: built, default on (ADR 0477).

TAKEAWAY A payload byte is copied once on receive and then travels by reference to the store and to every replica.

Cut-through: fan-out removed, receiver pre-write on by default

- **Why**: a 4 MiB write at QD1 is a store-and-forward chain (client frame, repop frame, replica store write, ack); from QD8 up the drive is full, so only low-depth latency is open.
- **Fan-out cut-through** (primary forwards the repop while the client frame still arrives): built, measured, **removed** (ADR 0061): it won at QD1 and lost at depth and in mixed load, because a forwarded frame holds the peer link and the PG for the client's fill time.
- **Receiver pre-write** (ADR 0477): the receiver writes the payload pieces to reserved free blocks while the frame still arrives, so the store time hides behind the receive. **Default on** (`flashstore_prewrite` true); `false` stays a runtime switch for diagnosis. No on-disk format or wire change; an unclaimed reservation is released, a crash leaves free blocks.
- **Safety**: nothing references pre-written blocks before the whole frame is verified; a piece is written only into free blocks, only when the device window admits it and only for a session whose caps allow writing the pool.
- **Tail**: rep3 depth stalls on VM labs are a receive-side interrupt stall (AMD AVIC) and a same-host replica link (attribution unproven); not seen on one bare-metal knet / kernel-NVMe run, proof pending for SPDK and RDMA.

OPEN GATE ITEM	STATE (ADR 0477)
Tail	Bare-metal tail proof on knet, SPDK and RDMA cells: pending; same-host replica link (attribution unproven)
Depth CPU	First-piece wake of a frame whose PG core is not its connection core; lazy wake in place
EC	No pre-write for EC sub-writes (deferred)

Sibling paths: knet and RDMA (feeds the decoder by copy) pre-write; SPDK and kernel NVMe share the store side. EC, compressing pools, secure-mode connections and frames below one piece: not candidates. Bare-metal QD1 / QD8 4M and EC cells: pending.

TAKEAWAY **Pre-write hides the store time of large writes behind the receive; the tail proof on bare metal is open.**

STATUS default on · ADR 0477 · fan-out removed (ADR 0061).

EC: overwrites need a read round, full writes and reads one pass

CASE	PROTOCOL (CLASSIC WIRE, ADR 0049)	PAYLOAD MOVEMENT
Partial overwrite (RMW)	Read round: the primary reads the old data and parity chunks (its own shard in parallel with the sub-reads). Write round: encode, sub-write per shard (transaction, log entry, rollback stash of the old extent). Roll-forward once every shard has committed.	Sub-read replies kept as shared buffers; sub-writes carry data as frame parts, crc only, no copy (ADR 0113). k+m-1 sub-writes go to the other shards.
Full-stripe write	No read round: plan and encode on the primary PG core, one sub-write per shard, parity buffers zeroed and filled in place.	rx copy + crc; gather copy fused with crc; one fused DMA copy per shard. The long plan-and-encode span runs on a core that also receives.
Healthy read	The primary asks its own and every remote shard in one pass; the local shard is addressed by chunk offset, no logical-offset probe (ADR 0116).	Shard sub-read replies by reference (store / prefetch buffers, no copy; ADR 0116), crc hints; interleave fused; reply as parts.
Degraded read	Surviving data and parity shards, decode per missing column. No timer gives up a sub-read: a source is dropped only when a map shows its OSD down or restarted (ADR 0049).	Zero-filled output buffer, decode over each missing column.
Optimized EC (mode 0)	Pools with `allow_ec_optimizations` are served by default (ADR 0067). Non-primary shards never lead. The per-PG extent cache is on; direct shard reads and parity-delta writes are implemented and off (`ec_optimized_direct`, `ec_pdw`) until their own A/B.	Parity delta (`ec_pdw`, implemented, default off) reads only the overwritten range.

STRUCTURAL COST
An RMW overwrite is two network round trips and one device read (classic protocol); replicated writes need neither.

MATH
Own GF(2^8) code checked against the upstream corpus (ADR 0048); AVX2, NEON and portable kernels; no ISA-L.

OPEN
Parity buffers are zeroed and allocated per op; the rollback stash costs metadata only (shared extents, ADR 0021 S10).

TAKEAWAY Reads and full writes are one pass; overwrites pay two round trips and a device read by protocol.

Classic-compatible peering, checked against a model of classic peers

- **Peering, recovery, backfill, scrub, snap trim, split and merge** follow the classic protocol, so classic OSDs, mons and clients see no difference.
- **Executable classic-peer model** (ADR 0050): every Sim run checks each message to a classic node against the spec's state × event table, and fails on one a classic PG would abort on.
- **Mixed acting sets** (classic + croit-flashstore-osd) are the migration path and a hard requirement (ADR 0013). Replicated pools size 1-3, min_size per pool.
- **A replica never refuses a PG transaction** (ADR 0066): data waits in waves behind the device queue; a refusal is fail-stop, never a silent drop.
- **Damaged local object = missing** (ADR 0065, replicated pools): a read error or bad object_info is repaired from a peer, as classic does.
- **Recovery, backfill**: classic reservations and push/pull; a replaced OSD is backfilled from its peers, EC rebuilds a lost shard from the survivors (ADR 0049). Cluster sessions stay lossless across socket loss while the peer is up (ADR 0071).
- **Full, release**: failsafe drop and admission, never a crash (ADR 0021); boots only inside the supported release window (ADR 0045).

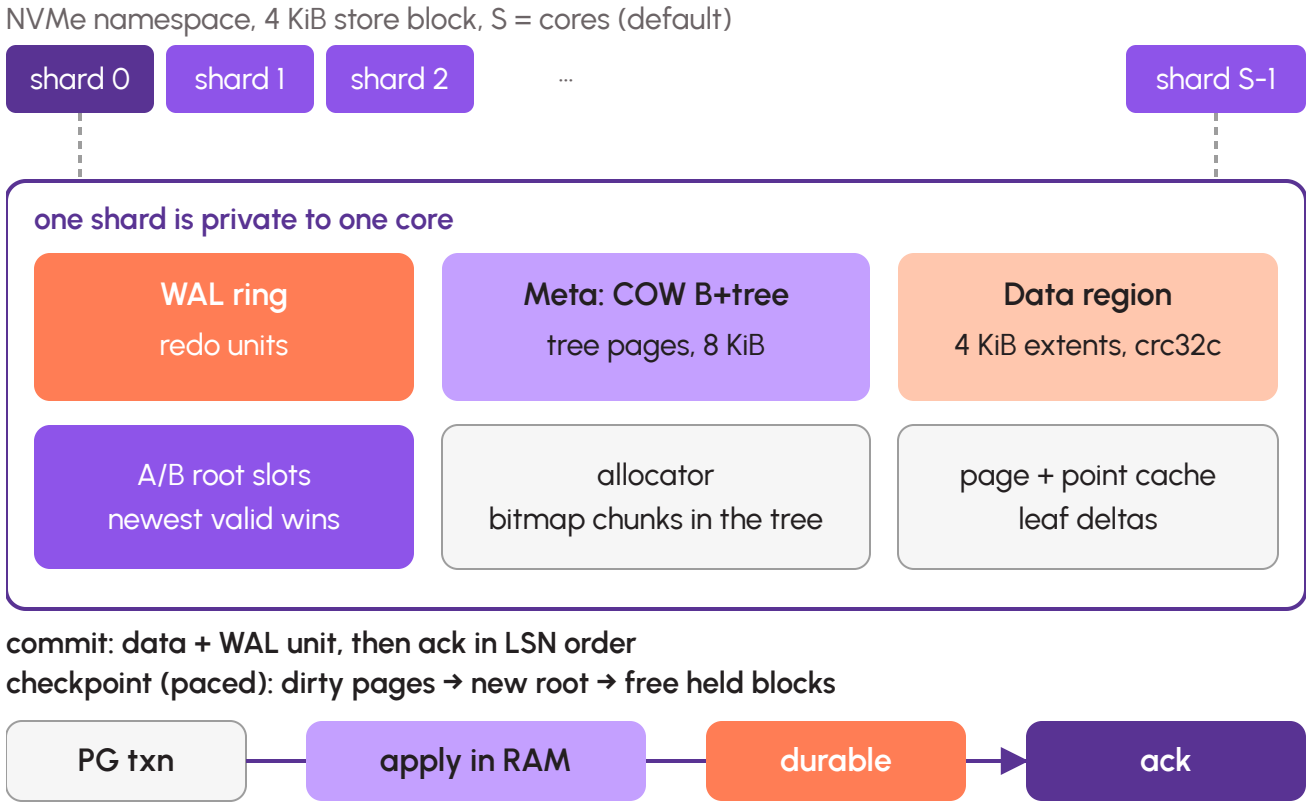
CONCERN	MECHANISM	ADR
Interop safety	One outbound choke point `PgSet::finish`; split gate matched per pgid, shard-agnostic (a shard-specific match made classic OSDs abort on split-child notifies; regression test).	0039, 0050
Split / merge	Children keep the parent shard. Cross-core merge = durable streaming copy, generation key for the crash window.	0031
Snapshots, copy	SnapMapper, trim step, purged_snaps; copy-from / copy-get also in mixed acting sets.	0037, 0057, 0058
Scrub	Resumable deep-scrub slices read fresh bytes and yield the core. Replica reservations in ascending (osd, shard) order like classic, no reservation timeout; an unfinished session keeps its operator request. Client writes preempt a chunk a bounded number of times; `pg repair` bypasses the limits.	0083
Client backoff	Down, incomplete or inactive PGs and unfound objects answer BLOCK instead of parking; a backoff record in DELETING state still drops ops in its range until the client's late ACK_BLOCK.	client-ops s6

Reservation budgets (`osd_max_backfills`, scrub slots, trimming PGs) are OSD-wide, kept by core 0 (ADR 0031, 0083). **Open**: a PG of a pool created while no OSD existed stays unknown.

TAKEAWAY **Wire-compatible with classic peers, and tested against a model of them in every run.**

Redo WAL plus copy-on-write B+tree per shard, no RocksDB by default

- **S = cores shards** per device by default (ADR 0160), fixed at mkfs, one WAL ring per shard; a core owns the shards s mod cores. Core 0 places every arriving PG (ADR 0639); the hash (pool + seed) mod S is only the fallback. A core owns whole shards, so there are no cross-shard locks (ADR 0021).
- **Per shard:** WAL ring, meta region (tree pages), data region, A/B root slots, own allocator.
- **One COW B+tree** holds collections, onodes, xattrs, extent maps with per-4k crc32c, omap, PG log, refcounts and allocator chunks. 8 KiB pages.
- **Data is never overwritten in place** and never journaled (small inline objects excepted): fresh 4 KiB extents. Clones share extents by reference (inline or heavily fragmented sources are copied).
- **Mount** = newest root slot + bounded WAL replay. Small objects live inside the tree.
- **Compression is per-pool opt-in** with BlueStore semantics, default off, zero cost when off (ADR 0059, compression page). OMAP sits behind a KV seam: B+tree is the default, opt-in engines at mkfs (omap page).



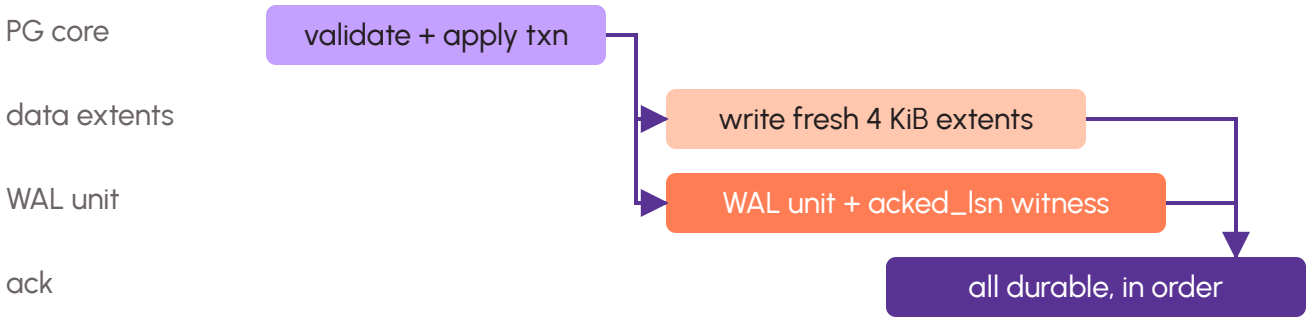
How to read: one shard is private to one core. A txn is applied in RAM, then made durable (data and WAL in parallel), and only then acked, in LSN order. Checkpoints run later and paced.

TAKEAWAY **Data and WAL go out in parallel; no LSM compaction to pay for.**

SNAPSHOTS a COW clone of a fragmented 4 MiB RBD head shares extents instead of copying. **GROUP COMMIT** shares device blocks.

Acked means durable: parallel commit, ordered acks, safe replay

- **Parallel commit:** data extents and the WAL unit are submitted together. The unit carries a crc of every new data block (ADR 0021).
- **Ack in LSN order,** only when this txn and all earlier txns are durable. A WAL unit is validated and acked atomically.
- **Group commit:** under load a unit is always in flight; at QD1 an idle shard submits at once.
- **PLP required:** completion = durable, no flush. Without PLP (lab only, `croit-flashstore-tool mkfs --allow-vwc`) every write is synchronous (FUA or RWF_DSYNC).
- **Freed blocks are held** until a checkpoint passes the freeing record.
- **Checkpoint freeze in slices:** a bounded number of pages per core poll, txns commit in between; crash safety unchanged.
- **Replay** validates data crcs after the last checkpoint. A bad unit ends the log, unless a later unit proves it was acked.



How to read: read down the rows. Data and WAL start together after validation; the ack waits for both and for every earlier txn.

FAILURE	HANDLING
Power loss, torn tail	Only unacked units can be damaged; the log ends at the first invalid unit. Acked data is not lost; media integrity is the device's job (ADR 0008).
Bit rot in an acked unit	A later valid unit's `acked_lsn` proves the cut txn was acked. Mount fails (`WalLost`); the OSD does not boot and PGs recover from replicas. Override: `--accept-wal-loss`.
Data block crc mismatch	Replicated: healed from a peer; EC: replanned around the shard, EIO below k good shards. A peer copy is never deleted for a local read error (ADR 0042, 0065).
Replica store refuses txn	Fail-stop; the primary re-runs without us (ADR 0066).

TAKEAWAY **A torn tail loses only unacked work; proven acked damage stops the OSD instead of rolling back.**

DIRTY COUNTS are bounded per shard. COUNTERS per core in `perf dump`.

One space for tree pages and data; reserves keep deletes working

MECHANISM	WHAT IT DOES	REF
Per-shard allocator	Next-fit on 4 KiB blocks; bitmap chunks flushed at checkpoint freeze; no cross-shard contention.	0021
Elastic meta space	A small base plus ~1 MiB cells from the data allocator, returned when empty, so omap-heavy pools grow meta on demand.	0021
Admission, not failure	`write_room` checks data and meta room before a client write: plain -ENOSPC, nothing logged or replicated. Omap and xattr batches count entries, not bytes. Deletes and truncates always pass.	0021
Reserves	Three tiers: user txns stop at a floor; system txns (osdmaps, PG state, repops) use a small hard reserve below it; pure shrinks use everything.	0021
Honest fullness	statfs reports at least the fullest shard, so mon full ratios fire before a hot shard runs out; checked every tick.	0021
Recover from full	Delete headroom and recovery without restart; deletion at NoSpace never fails a PG start or re-create.	0021
Shard capacity	Elastic pool: an object can outgrow its base share; without it, its shard's share.	0021 S14
Elastic shards	Default on for production drives: a minority of data space static per shard, the rest a shared pool lent in 1 GiB chunks on demand (next pages). Borrowed chunks are reclaimed.	0021
Device expand	Offline, crash safe; `croit-flashstore-tool expand` (refused on chunked stores) and `free`.	0021

SIZING

Tree space grows with the object count: small per RBD object, larger with attrs and omap.

DESIGN RULE

Meta and data draw from one space, and statfs reports the binding resource, not the data fill alone.

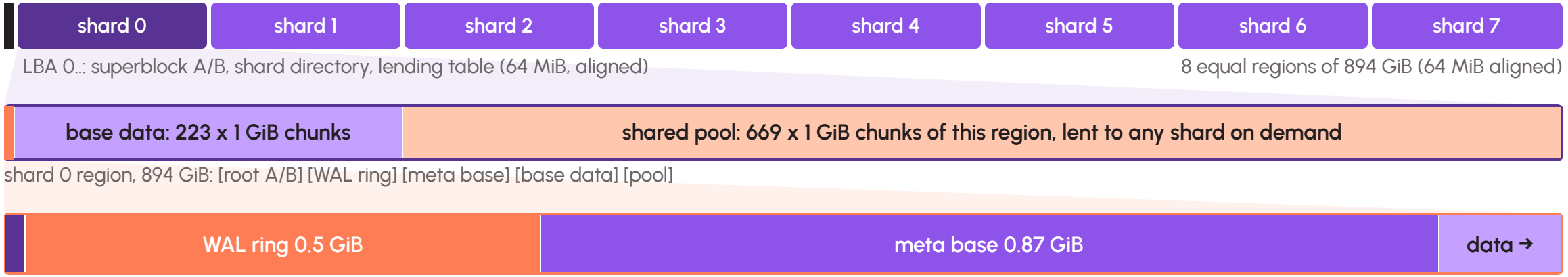
TRIM

On where the device can deallocate (probed at start): whole free windows are deallocated in device idle gaps and held until the completion (ADR 0642).

TAKEAWAY A full store refuses new writes and keeps accepting deletes; it never crashes.

One NVMe, N shards: every core owns a private slice of the drive

One OSD = one namespace, 7.68 TB = 7,152 GiB, 4 KiB block. Example only (values from layout.rs): N = 8 shards (= cores, PG-per-core ownership). Bars are to scale.



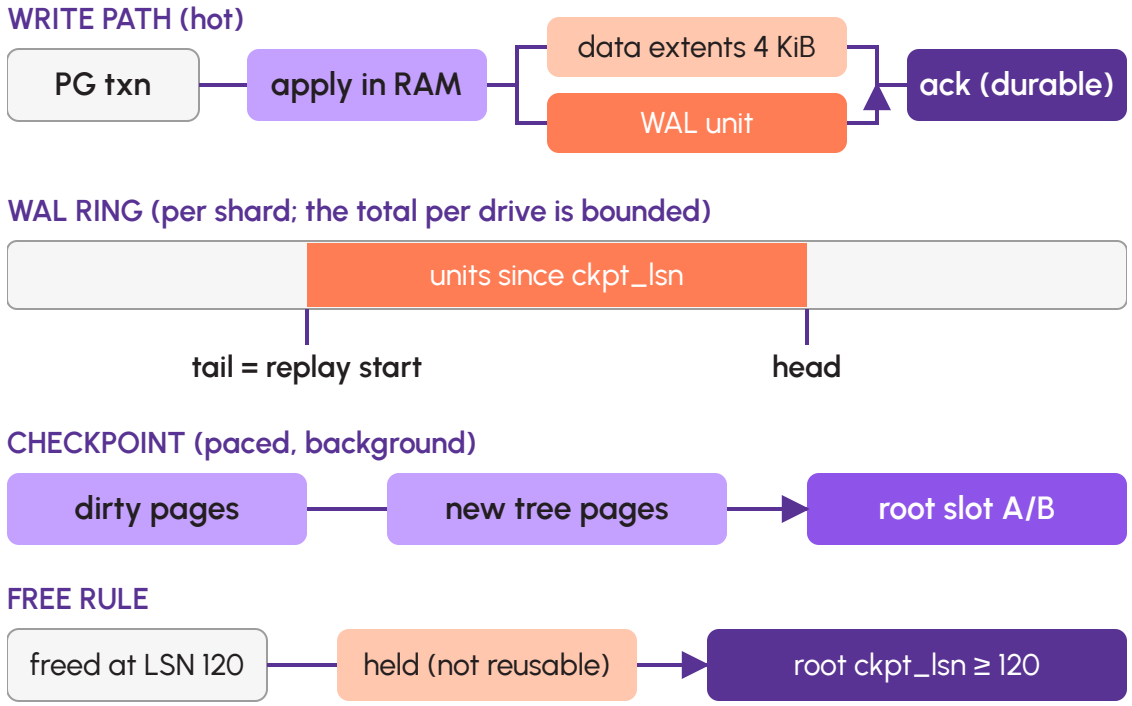
DRIVE · N SHARDS	REGION	WAL RING	WAL TOTAL	META BASE	BASE + POOL CHUNKS
Example: 7.68 TB · 8	894 GiB	0.5 GiB	4 GiB · 0.06 %	0.87 GiB	223 + 669

Formulas (`layout.rs` , ADR 0160, ADR 0021 S14)
region = (drive – 64 MiB head) / N, floored to 64 MiB
WAL ring = clamp(min(4 GiB, drive/64) / N, 64 MiB, 4 GiB)
meta base = max(64 MiB, drive / N / 1024)
base data = a fixed fraction of the whole chunks left; the rest is pool (at least two chunks per shard)

TAKEAWAY **Fixed areas are a small fraction of the drive plus a bounded WAL; the rest is data or lendable pool.**

Per shard: COW B+tree, redo WAL ring, paced checkpoints

- **One COW B+tree** (8 KiB pages) holds onodes, extent maps with per-4 KiB crc32c, omap, xattrs, PG log, refcounts and the allocator bitmap. Small objects sit inline in a leaf.
- **Data is never overwritten and never journaled (small inline objects excepted):** fresh 4 KiB extents, new extent map entries in the tree.
- **Redo WAL:** variable-size units (64 B header, payload in whole blocks) in the shard's ring, crc-chained, each carrying the crc of its new data blocks. Group commit shares units under load.
- **Checkpoint,** paced (ring fill or a timer): dirty pages are written as new pages, then a new root slot (A/B, newest valid wins) records the tree root and `ckpt_lsn`. The ring tail moves up to it.
- **Held-until-checkpoint:** a freed block (old extent, old page) is not reusable until a durable root passes the freeing record. Replay can then never meet a reused block.
- **Mount** = newest root slot + replay of units after `ckpt_lsn`.

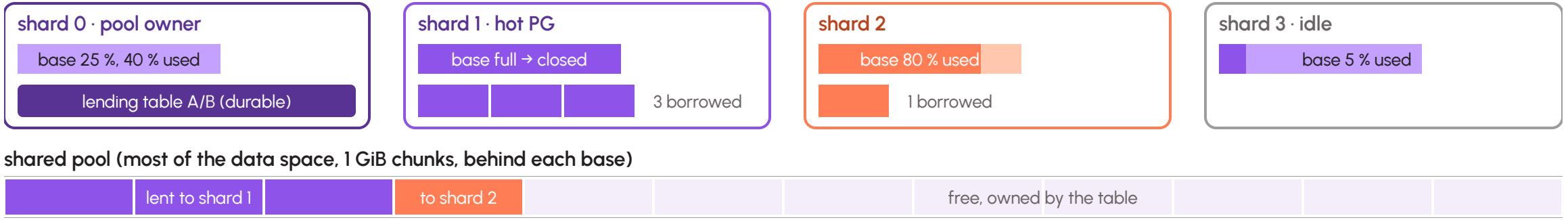


then the block returns to the shard allocator (next-fit bitmap, 4 KiB)
crash before the root lands: old root + WAL replay still see the old blocks

TAKEAWAY **Data and WAL go out together, the ack follows durability; space is reused only after a checkpoint.**

Static base per shard plus a shared pool of 1 GiB chunks

4 of N shards (one per core at the intended layout). Base = static, fixed at mkfs. Pool chunks sit behind each base; color = borrower, light = free.



borrow: below watermark → request to core 0 → table block written (durable) → grant → chunk joins the allocator
return: chunk empty → hole → free entry written → pool-grew signal to denied shards

WHY: STRANDING

A PG lives in one shard, so a fixed slice strands space: a shard with few PGs stays empty while a hot shard fills. With elastic shards the OSD fills evenly and reaches the mon `full` ratios like a classic OSD.

BORROW: EXACTLY ONCE

Below a watermark (whole free windows) a shard asks core 0, the only table writer. The grant is durable before use; messages cross cores once (SPSC lane). A full base closes and reopens with hysteresis; borrowed space is next-fit.

RECLAIM

An empty borrowed chunk goes back once nothing is allocated, reserved or held in it and the shard keeps its own headroom. Live data moves by COW rewrite only on demand, while another shard waits for a chunk. Chunks pinned by shared extents, external omap values or a meta cell stay.

TAKEAWAY Cores keep private bases; hot shards borrow chunks through one owner, durably.

Space per byte stored: small fixed areas, no per-object waste

DATASET	SPACE DESIGN
RBD image	Fresh 4 KiB extents, never rewritten in place; the extent map with a crc32c per 4 KiB is comparable to BlueStore csums. Snapshots share extents.
Small objects	Small objects live inside the tree and take no 4 KiB data block; attrs sit in the onode.
Omap	Keys and values in the shared tree, large values in data blocks. Inline user omap values are zstd-compressed with trained per-class dictionaries (`FEAT_VAL_COMP`, ADR 0021, default on); keys, xattrs and PG logs are not.
Held blocks	Freed space counts as free in statfs; a checkpoint runs before ENOSPC.

Space is one discipline of the never-worse-than-BlueStore rule.

FIXED AREA	SCALES WITH
Head (superblock, directory, lending table, alignment)	constant, negligible
Meta base	the shard region (device / N / 1024), a small fraction
Region alignment	shard count, at most one alignment unit per shard
WAL rings (ADR 0160)	a bounded total per drive (a fraction of the device up to a cap), split over the shards

- **Fixed areas**: the WAL rings, bounded in total per drive and split over the shards.
- **Per object**: small inline objects take no 4 KiB block; extent maps with per-4 KiB crc are comparable to BlueStore csums.
- **Checkpoint before ENOSPC**: held blocks are released by the next checkpoint, and a miss at ENOSPC forces one.

TAKEAWAY **The bounded WAL is the one fixed area; space is measured as bytes used per logical byte.**

Huge omap (RGW index) without stalling the PG core

PROBLEM	WHY IT COSTS	MECHANISM AND RESULT (INSIDE THE SHARED TREE, NO SECOND ROOT PER OBJECT)
Range delete, clear or remove of a huge omap	Cost grows with the key count; a huge cold tree would block the core for a long time.	**Lazy free (`FEAT_LAZY_FREE`):** covered subtrees are retired by address; non-resident interior pages go to a dead list kept in the checkpoint. Cost = the two boundary paths. Background drain with bounded reads in flight.
Checkpoint write amplification	A small random omap update dirties its leaf and parents.	**Leaf deltas (`FEAT_LEAF_DELTA`):** a small change is a delta entry in a shared page; the leaf and its parents stay clean. The delta cache is bounded per memory scale unit.
Removing keys	Per-key counters would need a read of every old value and a pre-count.	**Blind removals (ADR 0647):** no per-object omap counters; a range removal is one tree range delete, a large key batch a run of removal entries; no pre-count, no old-value read. Usage comes from deep scrub (ADR 0614).
Cold omap scans	Tree misses must not spin the PG core.	Non-blocking pass refuses cold pages, loads them, reruns the op; listings use rolling readahead.
Parked txns behind a cold head	A cold head parks the shard FIFO.	**Lookahead over the parked txns** loads their name and object pages in parallel; O(1) park index.
Values in the tree	A fragmented value dirties a page per op and travels in the WAL record.	**Larger values in data blocks** (FEAT_EXT_VALUE, mkfs); the tree keeps a small reference.

KEY LIMITS

A key must fit a tree entry; big values live in data blocks. Inline omap values are compressed by default (`FEAT_VAL_COMP`).

CRASH SAFETY

Dead list and deltas sit in the checkpoint; crash matrices cover every IO boundary.

SEAM STAYS OPEN

KV trait. B+tree default; LSM (ADR 0610) opt-in at mkfs; RocksDB (0681), SplinterDB (0727): proposed, not in prod builds.

TAKEAWAY Deleting a huge omap does not stall the PG core; big values live outside the tree.

One memory envelope from osd_memory_target, a soft target

- **One knob** T = `osd_memory_target`. Ingress I = a fraction of T, split client / peer. Caches = T – `osd_memory_base` – I (at least `osd_memory_cache_min`), rebalanced between shards by demand (ADR 0062, 0175).
- **Soft target**: a frame reserves its length from its pool at the header; a full pool parks the connection there (TCP backpressure, FIFO, no drop, no timer). Operations never fail or stall on memory, they slow smoothly (ADR 0175).
- **Per shard**: pages (8 KiB), point cache, leaf-delta cap and checkpoint arena share one scale unit, with a minimum tree size.
- **Read once at start** from the local conf, `--set` or the mon config database; `ceph config set` applies at the next restart.
- **Outside the budget**: RSS is not enforced. Hugepages and the RDMA send / receive arenas (own budget) sit outside it, opt-in RDMA read arenas inside; the PG log and recovery push estimates are inside the base, not measured.
- **QoS options are behaviour**: mclock profiles, wpq and the cut-off act as per-core admission shares (ADR 0752, Op QoS page).

CONSUMER (DUMP_MEMPOOLS)	SIZING / ACCOUNTING
`nvme_tree_pages`	resident 8 KiB pages, CLOCK eviction
`nvme_point_cache`	hot key cache
`nvme_delta_chains`	leaf-delta cap per scale unit
`nvme_alloc_bitmaps`, `nvme_ckpt_arena`, `nvme_wal_units`	allocator, checkpoint copy, WAL buffers
`osd_pglog`, `buffer_anon`	classic pool names (PG log is an estimate per entry)
`ingress_client|peer|held`, `envelope`	pool occupancy, limits and stalls; target, base, ingress, caches
`read_shared`	read buffers pinned by reply queues, sends and retained prefetch runs
`process`	VmRSS / VmHWM from /proc/self/status

GATE

Lab case 20: many concurrent large writes keep the process peak near the target.

OPEN

PG log of a degraded OSD with many PGs, recovery push bytes and byte-relay pipes are estimates inside the base.

TAKEAWAY **The target sizes ingress and store caches and brakes smoothly; RSS also holds hugepages and network buffers.**

One store, two device tiers: kernel NVMe in production, SPDK optional

ASPECT	KERNEL NVME (PRODUCTION)	SPDK (PROD-FULL BUILD)
Device spec	`kernel:/dev/nvmeXnY` = O_DIRECT on the block device; `kernel:/dev/ngXnY` = NVMe passthrough (`IORING_OP_URING_CMD`) on the generic char device (ADR 0169).	`spdk:BDF[,ns=N]`: controller on `vfio-pci`, userspace driver (ADR 0004).
Queues	One io_uring per core thread, shared by all queue handles; registered device fd and a registered buffer arena (no per-IO page pinning); one `io_uring_enter` per loop pass.	One polled qpair per core thread, hugepage DMA buffers.
Waiting	The core sleeps in its ring and a completion wakes it (one wait point with the network ring), as BlueStore sleeps on the interrupt. Hybrid park (`--knet-store-park`) keeps a short poll.	The core polls completions; a QD1 read spins for the device wait.
Durability	Both: PLP required, completion = durable, no flush on the hot path; without PLP (lab only, `mkfs --allow-vwc`) FUA. Geometry checked from Identify: 4096 B LBA, no metadata, no PI, on both tiers; any other format is refused with a reformat hint (ADR 0008, 0169).	
Exclusivity	`O_EXCL` open of the namespace block device (refused when mounted or claimed by md/dm; LVM only as in ADR 0646).	Explicit PCI allow-list for every SPDK init; port probe before bind.
Selection	Backend follows the `--dev` spec stored at mkfs; `--store-backend spdk|kernel|kernel-cmd` overrides (an `spdk:BDF` spec with `kernel` resolves the kernel node from sysfs). The production binary carries the kernel tier only; `prod-full` adds `spdk` (ADR 0682).	

WHY THE KERNEL TIER

Polled completions make QD1 read CPU per op the device wait; the kernel tier sleeps.

STATE

Kernel NVMe is the production path (ADR 0169). Open: QD1 read CPU per op vs BlueStore. Native namespaces deallocate free windows (ADR 0642); the LVM case does not.

NAMESPACES, ONE LVM CASE

Whole namespaces, or one validated linear LVM volume on one partition of one namespace (honest geometry, no volatile cache); other mappings are refused (ADR 0646).

TAKEAWAY One store, two device tiers: kernel NVMe sleeps and is the production tier, SPDK polls; both ack only when durable.

Per-pool compression: BlueStore semantics, off by default

- **Status:** built, opt-in per pool, default off (`compression_mode none`); ADR 0059. The file name still says "no compression": the decision was superseded by the 100 % classic-parity mandate.
- **Semantics as classic:** `compression_mode` none | passive | aggressive | force, algorithm snappy | zlib | zstd | lz4, `required_ratio`, min / max blob size from the pool options; alloc hints scoped to the transaction. RGW compression stays in the gateway.
- **Layout:** a chunk is a logical range aligned to `max_blob_size` (a fixed number of whole blocks by default); the extent carries a COMPRESSED flag, algorithm, logical span and a logical crc for deep scrub; the per-block crcs cover the physical blocks.
- **Chunk windows** (ADR 0059; needs `FEAT_SHARED_EXT`): a mutation inside a compressed chunk writes only its own blocks. The cut record becomes up to two window records that share the chunk's physical blocks through extent refcounts; nothing is read, decompressed or recompressed, and the last owner frees the chunk. When the windows of an object cover too little of a chunk, the live part is rewritten so space does not grow.
- **Cheap exits:** an all-zero chunk is never stored; large chunks are sampled first and skipped when incompressible; replicas compress independently, the compressed form never goes on the wire.

ASPECT	STATE
Measured	pending (fresh BlueStore vs croit-flashstore-osd comparison of ratio and space after overwrites on the current build; benchmark deck, compression section).
Losses	Not re-measured since the chunk windows replaced read-merge-write overwrites (ADR 0059).
Not covered	The pool switch does not govern omap values (store feature `FEAT_VAL_COMP`, mkfs default on) nor xattrs and PG logs; a mutation that leaves too little of a chunk referenced rewrites its live part.
Where	`store::compress`, `obj::stage`; small per-shard decompressed-chunk cache; data in the benchmark deck, compression section.

TAKEAWAY **Compression is a per-pool switch with classic semantics; overwrites inside a chunk no longer recompress it.**

STATUS opt-in · ADR 0059 (chunk windows) · fresh comparison pending.

EC: on by default for allowed profiles, gated per pool and role

- **Allow-list gate** (ADR 0039), per pool on every map change: plugins isa, jerasure, lrc, shec and clay with the profile as the mon stores it; pool size, stripe width and chunk alignment must match the profile; append-only (L0), `allow_ec_overwrites` (L1) or optimized (mode O, ADR 0067: isa, or jerasure `reed_sol_van`; chunk a multiple of 4 KiB; not clay, lrc or shec). Anything else is declined.
- **Defaults:** `ec_enable_pools=all`, `ec_enable_primary=true`, `ec_enable_optimized=all`, `ec_enable_optimized_primary=true`. An operator opt-out (`off` / `false`) makes the OSD silent-passive, which stalls classic PGs.
- **Decline mode:** an unsupported EC pool gets the empty-shard answers a classic primary expects and an alert (`FLASHSTORE_EC_POOL_NOT_SERVED`); a pool with local data is never declined.
- **Shard role** under a classic primary (ADR 0040): sub-write and read, rollback, stash, push, scrub digests.
- **Primary role** (ADR 0049): crate `ecback`, own GF(2^8) math (ADR 0048), EC peering, recovery and backfill by shard rebuild, deep scrub.

PIECE	STATE
Shard role, live	Lab cases with classic primaries: basic and overwrites, kill under load, scrub corruption, failover, split, merge, empty objects. Checked for classic restarts and aborts.
Primary role	Sim scenarios: kill primary at random points, wiped OSD in several geometries, scrub flip in every shard. Live cases with classic shards (ec-phase-b.md).
Default gate	Kill/restart loops with no `ec_*` override, all-croit-flashstore-osd and mixed pools, green before the defaults flipped (ADR 0067).
Math	GF(2^8) kernels (AVX2, NEON, portable) checked byte-exact against the upstream corpus (ADR 0048).
Known gaps	No ISA-L; per-op buffers; direct reads and parity-delta writes (implemented) stay off (`ec_optimized_direct`, `ec_pdw`). Pool snaps (mksnap) on EC pools untested; self-managed snaps work.

Scope: the v1 charter is replicated pools (ADR 0013). EC is on by default because the core carries RGW and CephFS data pools (ADR 0010) and a passive EC member would stall classic PGs.

TAKEAWAY **A pool runs EC only when the gate allows it; the classic primary is never surprised.**

Prime rule: never crash or confuse classic OSDs and mons

OUTBOUND VALIDATION

Every PG message passes `PgSet::finish`: spg shard vs pool type, per-role allow-list for EC, default deny. Unsupported EC pools answer only as an empty shard (decline mode) or stay silent (ADR 0039, 0050).

CLASSIC-PEER MODEL IN EVERY SIM

State x event table from the spec, checked on each delivery to a classic node (ADR 0050). Found by it and pinned by tests: stray split children, RELEASE/REQUEST around ungranted reservations, EC from == primary.

RELEASE AND IDENTITY

Boots only inside the supported `require_osd_release` window (refused: exit 3, no hot restart loop). Purged or re-created id: clean exit 0 (ADR 0045).

FORMAT VERSION AND UPGRADE

The superblock carries a format version and feature bits; a newer version or an unknown bit is refused loudly (exit 78, store untouched). In-place upgrade: offline `croit-flashstore-tool upgrade [--dry-run]` with a crash-safe mark; one step exists (OMAP LSM); a version-1 store still needs re-mkfs; compat / ro_compat model proposed (ADR 0611).

TOOL AND DEVICE LOCKS

Offline tools exit 3 while the OSD runs (admin socket or device lock answers). SPDK carries an explicit PCI allow-list. Port probe before bind: no silent SO_REUSEPORT join (ADR 0053).

LEAST PRIVILEGE

Runs as user `ceph` with only CAP_IPC_LOCK and CAP_SYS_NICE ambient (the unit's bounding set is wider for smartctl; when started as root with `--setuser` the SPDK tier stays root with a warning; under the packaged unit it runs as `ceph` with vfio access). Container: all caps dropped, then only those its mode needs; read-only rootfs (ADR 0056, 0052).

Also: msgr2 mode (`ms_*_mode`) honored at runtime (ADR 0036); cephx caps; a CLIENT hello may carry a daemon ticket (the mgr's libcephsqlite handle authenticates as the daemon); device id and name equal classic Ceph's, so a drive migrated from BlueStore stays one device in `ceph device ls` (ADR 0490); the op scheduler options act as per-core admission shares (ADR 0752), and only options without a mechanism warn once. The RDMA banner bits are never required and never offered to mons, mgrs or heartbeat peers, so classic daemons see plain msgr2 v2 (ADR 0469).

TAKEAWAY **Every outbound message is checked in one place; unsupported pools get only the classic decline replies.**

The OSD tunes what it owns and only warns about the host

SELF-TUNED BY THE OSD

- Cores: default = CPUs in the affinity mask (at most the store's shards); L3-domain placement, NIC-interrupt CPUs avoided when enough others remain (ADR 0031, 0063).
- Store shards fixed at mkfs; queues, windows, receive helpers per core.
- Memory split of caches and ingress pools from `osd_memory_target` (ADR 0175).
- Idle policy: quiet cores sleep in their ring (ADR 0032, 0053); RDMA windows follow the per-core budget.
- Backend capability (io_uring, zero copy, zcrx) probed at start, never gated by kernel version.

OWNED BY THE HOST: WARN, NEVER CHANGE

- NIC offloads (GRO-hw), IRQ affinity, ring sizes, channels, PCIe link.
- Sysctls (rmem/wmem, somaxconn, NUMA balancing), THP, CPU governor; hugepages and vfio module (SPDK).
- zcrx NIC state (header-data split, flow rules).
- Kernel boot parameters: `init_on_alloc=0` (page zeroing and hardened usercopy cost CPU on large reads; hwcheck `os.init_on_alloc` warns with the grub / grubby command). No hardened-usercopy check yet (open).
- RoCE: MTU, PFC, DSCP trust, DCQCN (ADR 0470).
- Each warning at startup and in `--check` carries the exact command.

TOOL	WHAT IT DOES
`croit-flashstore-osd --check` `croit-flashstore-tool hwcheck`	Read-only host and hardware check (ADR 0166). Sections: Build, CPU, Memory, IOMMU/VFIO, NVMe, Network, RDMA, Kernel/OS, Config. Status OK / WARN / FAIL / N/A with value, reason and fix; reports which tiers are usable (kernel TCP, RDMA, kernel NVMe, SPDK). Exit 0 = no FAIL, 1 = FAIL, 2 = usage; `--format plain|json|json-pretty`.
`croit-flashstore-tool host-tune`	Applies exactly the mechanical fixes of hwcheck, on explicit request only (ADR 0465). Dry run is the default (current value, target, command); `--apply` records undo operations in a state file; `--revert STATEFILE` replays them; `--check ID` and `--iface IF` select; `--emit-persist` prints boot-persistent setup. Idempotent; needs root or the three capabilities (exit 3 otherwise). The OSD never runs it.

The printed warning and the executed command are the same data, so advice cannot drift from what host-tune does.

TAKEAWAY **Optimal by default for what the OSD owns; for the rest, one command per finding and an opt-in helper.**

SMART and devicehealth: classic ids and answers

- **Status:** built, on; ADR 0490, 0055, 0034.
- **What the OSD reports:** `smart` and `device_ids` / `device_paths` / `devices` metadata from the NVMe Identify Controller and the SMART / health log page; same on the SPDK and kernel tiers.
- **Device id = classic form:** model + `_` + serial from Identify, cut at the first NUL, padding kept, spaces as `_` (ADR 0490); a drive migrated from a classic OSD stays one device in `ceph device ls`. Device name: kernel tier = kernel block name, SPDK tier = PCI address.
- **mgr devicehealth works:** the libcephsqlite handle of the mgr authenticates as mgr.x and sends a daemon ticket in the msgr2 CLIENT HELLO; the OSD accepts it. Scrape and history of `get-health-metrics` are served.
- **hwcheck** (ADR 0166) shows the NVMe health fields with the PLP and LBA-format checks; the OSD never changes device settings.

LAYER	RULE
nvme::health	Identify + SMART log parse, `Ident::device_id`; shared by both tiers.
OSD	`ceph tell osd.N smart`, asok `smart`; metadata keys as classic.
mgr	devicehealth scrape every interval.

TAKEAWAY Drives register once, in the classic id form, and the mgr devicehealth module is served.

STATUS on · ADR 0490 · daemon ticket in CLIENT HELLO accepted.

Deploys like an OSD: package, systemd or container, one binary

- **Host**: kernel tier: io_uring, a NIC the kernel drives, memlock unlimited. SPDK (prod-full) adds IOMMU, `vfio-pci`, hugepages. `--check` reports the rest.
- **Native**: `croit-flashstore-tool prepare --dev kernel:/dev/nvmeXnY --id N` (or `spdk:PCI`), then `systemctl enable --now croit-flashstore-osd@N`.
- **Packages** (ADR 0589): `croit-flashstore-osd`, `librustyrados`, `librustyrbd` (+ `croit-rbd`), among others; Debian, Ubuntu, Alma; the `prod` profile.
- **Container** (ADR 0052): debian slim, stripped binaries, no ceph packages; the `prod` binaries need only glibc. Healthcheck via admin socket; podman unit.
- **Exit codes** (ADR 0045): 0 = clean stop or purged id (stays down); 2, 3 and 78 = usage, refused or store needs upgrade / newer than the build (stays failed); other codes = crash (restarted).
- **Shutdown** (ADR 0034): SIGTERM marks the OSD down; exit 0. Rolling upgrade: one OSD at a time.
- **Tools** (`croit-flashstore-tool`): prepare, mkfs, activate, list, inspect, asok, fsck, repair, upgrade, sizes, expand, free, objectstore, raw, dump, support-bundle, wal-inspect, hwcheck, host-tune, nvme-rebind, among others.

HOST NEEDS PER TIER

TIER	HOST NEEDS
all tiers	glibc 2.34 or newer (Alma 9 floor), memlock unlimited
knet (kernel TCP)	io_uring (`kernel.io_uring_disabled`, container seccomp); any NIC the kernel drives
kernel NVMe	namespace on the kernel `nvme` driver, 4096 B LBA without metadata or PI, PLP; `O_EXCL` open
RDMA (opt-in)	RoCEv2 netdev with RDMA device, `ib_uverbs`; `RLIMIT_MEMLOCK` covers `flashstore_rdma_memory`
SPDK (prod-full)	IOMMU, `vfio-pci` (or uio), hugepages, `libnuma1`, `libelf1t64`

Not verified without bare metal: vfio NVMe passthrough into the container (SPDK), cpuset and NUMA effects, read-only rootfs with the SPDK environment.

TAKEAWAY **Same binary and CLI under systemd and containers; exit codes tell the supervisor whether to restart.**

CLIENT LIBRARY

rustyrados: the native Rust client, event-driven by default

- **Scope** (ADR 0306): Ceph v20.2.4 and v21.2.0, each version's public ABI and wire protocol, separate manifests and evidence. Library `rustyrados`, packaged as `librustyrados`, activated via `ld.so.conf` (ADR 0450, 0589).
- **Compat policy** (ADR 0256): ABI, ownership, completion, ordering and durability contracts are mandatory; fixes may differ from librados where application and wire compatibility hold.
- **Transport default:** event-driven kernel TCP reactor (epoll) with efficient idle: no core is held while idle (ADR 0402). io_uring is opt-in (`rustyrados_transport_backend = epoll|io_uring|auto`), chosen by probe, never by kernel release (ADR 0466). DPDK client transport removed (ADR 0682).
- **One protocol owner** per client; receive copy and crc of large reads on helper threads, sends stay on the driver (ADR 0467).
- **RDMA:** the client upgrades OSD sessions with the same msgr2 handshake; large writes can leave zero copy from a registered payload pool (ADR 0469). READ placement: lab opt-in, disabled in production (ADR 0471, proposed).
- **Time:** internal time is 64-bit; the wire keeps u32 seconds, saturated at one boundary function (ADR 0747).

LAYER	ROLE
`rustyrados`	Native Rust API with owned payload buffers; request engine, placement, auth and retry semantics independent of the transport.
`rados-rt`	Runtime: reactor, msgr2 sessions, transport drivers (epoll, io_uring, RDMA carrier), receive helpers.

Status: v21.2.0 ABI package gate passed (C and C++ symbols, none missing); full ABI of v20.2.4 open; not release-qualified. Performance against librados: pending (fresh same-session A/B). Open: the remaining gap to librados; the default switches to `auto` once io_uring is at least epoll in every cell of the same-session A/B.

TAKEAWAY **A librados-compatible client that idles cheaply by default and shares the OSD's RDMA upgrade.**

librustyrbd: the native librbd, on top of rustyrados

- **Target:** the complete public Ceph v21.2.0 librbd C and C++ API, a native Rust API and unchanged QEMU RBD use. The image APIs are on in the default build; the C export set is complete in an all-features build; the C++ exports (ADR 0704) are implemented behind the non-default feature `librbd-cxx`. **Not release-qualified**.
- **Layers:** `crates/rustyrbd` (image, snapshot, parent and metadata queries, object map, journal), `tools/croit-rbd` (operator CLI). Own `libradosstriper` (ADR 0709).
- **Migration:** prepare, execute, commit and abort with the classic source and destination header contract (ADR 0705, accepted). **Format 1:** the `rbd_directory` object uses TMAP ops on the OSD, the header a plain write (ADR 0013).
- **Mirroring:** mirror resync and peer bootstrap calls are exported; journaling is specified, not accepted (ADR 0700, proposed). **Encryption:** encrypted images, with compare-and-write over the encrypted source.

AREA	STATE
C ABI	every C export present in an all-features build; symbol presence is not behavioural coverage
C++ ABI	implemented behind the non-default feature `librbd-cxx` (ADR 0704); not release-qualified
Migration	commit / abort accepted (ADR 0705)
Journaling, mirroring	journaling proposed (ADR 0700); mirror calls per spec
Evidence	CLI scenarios and recorded `cls_rbd` calls compared with classic; QEMU gate for the RDMA block path

Open: behavioural coverage of the full librbd API, QEMU qualification beyond the RDMA block path, behavioural qualification of the C++ ABI.

TAKEAWAY **RBD clients are our own: complete C export set, behaviour qualification still open.**

Layers of evidence: unit tests, crash matrices, simulation, labs

UNIT AND PROPERTY TESTS

Unit and property tests across crates, services and tools. Tree and store vs `BTreeMap` models (proptest), cold remounts and fsck; own frozen vectors for crc, CRUSH, denc, EC; store format golden keys per version. RDMA: seeded protocol model tests and soft-RoCE (rxe) integration tests.

CRASH AND FAULT MATRICES

Simulated NVMe with fault injection; crash at IO boundaries, second crash on the recovered store, fsck of every crash image. Recovered state equals the model after a txn prefix that holds every acked txn. Soaks only when durability code changes.

DETERMINISTIC SIMULATION

Seeded multi-OSD simulations of peering and ops, single-threaded by design, with the classic-peer model on by default (ADR 0050). The primary is killed at random points.

MIXED-CLUSTER LABS

Docker labs with classic mon, mgr and OSDs plus croit-flashstore-osd: numbered cases (msgr2 modes, restarts, EC, scrub reservations, memory envelope, connection kill, …), rbd, rgw and ostool suites. Most cases check classic logs for aborts.

REVIEWS AND PERF GATES

Risky code (WAL, lazy free, RDMA teardown) gets targeted reviews; RDMA has a fault campaign. A/B runs use interleaved rounds; one repeat per arm, three only when a delta lies inside the noise.

TAKEAWAY **Evidence is layered; parity is checked against classic with our own tests.**

LAB functional suites run on lab VMs; bare metal only for SPDK and performance.

PERFORMANCE METHOD

Never worse than BlueStore: every discipline is a gate

- **Gate (project rules, AGENTS.md):** latency, IOPS, throughput and space of the kernel tier at or above classic BlueStore on the same hardware; CPU per op is secondary (higher is fine when IOPS and bandwidth are higher) but measured. A cell that loses is profiled and fixed, not excused. Prefer the very fast option over "fast enough" unless it breaks compatibility.
- **Targets:** line rate per OSD on fast NICs; high single-stream throughput in msgr2 crc mode; cycles per byte per core as the metric; benchmarks at a small fixed core count per OSD.
- **Method:** interleaved A/B rounds and medians; per-stage latency and CPU of BlueStore against croit-flashstore-osd with flame graphs; hunt double copies; the same OSD-level scenarios on every sibling path.
- **Sibling paths:** every fix names the layer of its root cause and checks knet / RDMA, SPDK / kernel NVMe, replicated / EC, crc / secure msgr2. Same pattern in a sibling means fix it there and add a gate cell.
- **RDMA gate:** RDMA must be at least as good as knet in every measured discipline, with its own cycles counted, before it can become a default.

DISCIPLINE	GATE CELLS
Latency	4K read and write at QD1, tails (p99, p99.99), replicated and EC
Throughput	4K at depth, 64K, 4M read and write; single stream and many streams
CPU per op (secondary)	All busy cores of the OSD hosts, kernel TCP included; cycles per byte for large IO
Space	Fixed areas and bytes used per logical byte (RBD, small objects, omap)

Memory targets are soft: under pressure the OSD slows smoothly and never stalls or fails ops (ADR 0175).

TAKEAWAY **Speed is a standing gate on every discipline and sibling path; results are kept with their build ids, not on slides.**

STATUS LEDGER (1 / 3)

Every layer: state, default, ADR and the limit that matters

LAYER	COMPONENT	STATE	DEFAULT	ADR	KNOWN LIMIT / OPEN
Network	knet: kernel TCP over io_uring	production	on (only transport)	0053, 0054, 0464, 0682	zcrx receive probed per queue, opt-in until a bare-metal A/B; one TCP stream = one receive core
Network	RDMA (RoCEv2): upgrade of knet sessions	built, opt-in	off (flashstore_rdma)	0469, 0470, 0486, 0488, 0489	lossless fabric recommended, lossy works; READ placement (0471, proposed) off in production
Store	COW B+tree + redo WAL ring per shard	production	on	0021, 0160	versioned superblock; in-place upgrade: framework and one step (0611, proposed)
Store	Kernel NVMe tier (io_uring, O_DIRECT or passthrough)	production	on (prod build)	0169, 0646	4M fresh QD1 read CPU per op vs BlueStore; trails SPDK in busy cells; linear LVM volume accepted
Store	SPDK NVMe tier (polled)	built, optional	prod-full build only	0004, 0682	spins on the device wait: CPU per op of QD1 reads
Store	OMAP engines: LSM, RocksDB, SplinterDB	LSM built, opt-in; others proposed	off (B+tree default, mkfs choice)	0610, 0681, 0727	RocksDB and SplinterDB: non-default Cargo features, not in the prod builds; 0737 rejected; live measurement pending
Store	TRIM: whole free 1 MiB windows in idle gaps	production	on	0642	held until the completion; SPDK and kernel tiers
Store	Per-pool compression (BlueStore semantics)	built, opt-in per pool	off (compression_mode none)	0059	overwrites inside a chunk use windows, no recompression; omap values are compressed by the store (FEAT_VAL_COMP, ADR 0021)

TAKEAWAY **Production: knet, kernel NVMe, COW store, replicated and EC pools, op QoS, TRIM. Opt-in: RDMA, SPDK, compression.**

STATUS LEDGER (2 / 3)

Every layer: state, default, ADR and the limit that matters

LAYER	COMPONENT	STATE	DEFAULT	ADR	KNOWN LIMIT / OPEN
Store	Per-shard data cache (pinless, segmented LRU)	production	on (flashstore_store_pf_retain)	0043, 0633	bounded: per-shard floor plus unused tree page budget; scrub reads bypass; reads via a per-shard prefetch table
PG / core model	Thread-per-core, PG-per-core ownership	production	on	0006, 0031	one stream = one receive core
PG / core model	Per-object op ordering (gate)	production	on	0043	pure reads do not wait for each other
PG / core model	Op QoS: mclock profiles, wpq, cut-off as per-core admission shares	production	on	0752	shares per core; recovery budgets OSD-wide on core 0
Replication / EC	Replicated size 1-3, classic-compatible peering	production	on	0013, 0031, 0050, 0071	scope: flashstore-only clusters, mixed acting sets work with classic OSDs
Replication / EC	EC (isa, jerasure, lrc, shec, clay; optimized mode 0)	production	on for allowed profiles	0067, 0116, 0483	ec_pdw implemented, default off; no ISA-L; mksnap on EC pools untested
Cut-through	Fan-out cut-through (forward while the client frame arrives)	removed	-	0061	lost at depth and in mixed load (link + PG hold)
Cut-through	Receiver pre-write	production	on (flashstore_prewrite = true)	0477	bare-metal tail proof pending; same-host replica link (attribution unproven)

TAKEAWAY Production: knet, kernel NVMe, COW store, replicated and EC pools, op QoS, TRIM. Opt-in: RDMA, SPDK, compression.

STATUS LEDGER (3 / 3)

Every layer: state, default, ADR and the limit that matters

LAYER	COMPONENT	STATE	DEFAULT	ADR	KNOWN LIMIT / OPEN
Client library	rustyrados (librados-compatible)	accepted	epoll default, io_uring opt-in	0306, 0402, 0466, 0467	v21.2.0 ABI gate passed, not release-qualified; remaining read gap to librados at mid sizes
Client library	librustyrbd (librbd-compatible)	built	image APIs on in the default build	0700, 0704, 0705, 0709, 0745, 0748	C export set complete in an all-features build; C++ ABI behind non-default feature librbd-cxx; not release-qualified
Compat / observability	SMART + devicehealth (classic device id)	built	on	0055, 0490, 0034	mgr devicehealth served
Compat / observability	hwcheck + host-tune	production	on (warn); host-tune on request	0166, 0465	init_on_alloc checked; no hardened-usercopy check yet
Compat / observability	Sampled op trace + perf counters + admin socket	production	on	0170, 0055	–
Compat / observability	64-bit internal time, one wire boundary	production	on	0747	wire keeps u32 seconds

TAKEAWAY Production: knet, kernel NVMe, COW store, replicated and EC pools, op QoS, TRIM. Opt-in: RDMA, SPDK, compression.

What changed since 2026-10-28

AREA	CHANGE	REFERENCE
Network	AF_XDP and DPDK networking removed: knet is the only transport, RDMA upgrades its sessions	adr 0682
Store	Prod build = knet,mimalloc,rdma without SPDK; the backend follows the --dev spec (kernel:/dev/nvmeXnY, kernel:/dev/ngXnY, spdk:BDF); kernel NVMe is the production tier	adr 0682, 0169
Store	TRIM: whole free 1 MiB windows are deallocated in device idle gaps, held until the completion	adr 0642
Store	Validated single linear LVM volume restored for the kernel tier	adr 0646
Store	Blind omap removals: no per-object omap counters, range removal as one tree range delete	adr 0647
Store	OMAP engines beside the B+tree: RocksDB and SplinterDB (proposed; non-default features, not in the prod builds)	adr 0681, 0727

TAKEAWAY AF_XDP and DPDK networking removed; op QoS, TRIM, blind omap removals and linear LVM landed.

What changed since 2026-10-28

AREA	CHANGE	REFERENCE
Store	Downgrade safety: an older build refuses a newer store (exit 78); compat / ro_compat model proposed	adr 0611
QoS	Op scheduler options are behaviour: mclock profiles, wpq, cut-off as per-core admission shares	adr 0752
Cut-through	Pre-write takes the device window and checks the session caps before any device byte	adr 0477
EC	Legacy EC pools with FLAG_OMAP served (omap ops answer EOPNOTSUPP); stretch pools and wide isa cauchy profiles; all plugin families served	adr 0067, ec gate
Client	rustyrados: DPDK client transport removed; v21.2.0 replaces v21.1.2 as reference	adr 0682
Platform	64-bit internal time, one wire boundary	adr 0747

TAKEAWAY AF_XDP and DPDK networking removed; op QoS, TRIM, blind omap removals and linear LVM landed.

What DAOS does differently, and what we could still do about it

DAOS DOES	WHY IT IS FASTER	WHAT WE COULD DO (EFFECT)	VERDICT
Client computes checksums and EC parity; replicas pull payloads from the client by RDMA	Server CPU per payload byte near zero; leader NIC off the replica data path	Needs librados and wire changes. Fan-out cut-through was built and removed (lost A/B); receiver-side pre-write is built, default on (ADR 0477). RDMA between peers is opt-in (ADR 0469); READ placement is implemented but off by default (ADR 0471, proposed)	Not applicable to Ceph wire compatibility
Partial-stripe EC write without read-modify-write: data plus plain replicas to parity shards, parity aggregated later	A small partial write needs no read round (ours: a read round, a write round and a roll-forward)	Parity delta of optimized EC: read only the overwritten range, update parity by delta; wire-compatible. Implemented behind `ec_pdw`, default off (ADR 0067). Deferred parity: classic shards, recovery and scrub would misread it	Parity delta: flip the default after its A/B. Deferred parity: not applicable in mixed clusters
Background work scales with free space; freed extents returned to the device in paced batches	GC never steals poll time when space is ample; no discard burst after a mass delete	Free-space gauge for the share of free, defrag and scrub work; paced batched Deallocate after the hold: built as whole free windows in idle gaps (ADR 0642)	Gauge when near-full starvation shows; Deallocate done (ADR 0642)
Checkpoint is a DMA of dirty chunks (in-place image plus redo log)	Checkpoint CPU not on the PG core	Keep the COW tree (atomic root, clone sharing). The freeze already runs in slices (ADR 0021); page seal and crc on a helper core would remove the rest of the block	Candidate for the QD1 tail work
Peers address one target (core) directly	No head-of-line blocking across cores; a stream is not bound to one core	One msgr2 session per peer pair is fixed by the wire; a second connection replaces the session. In place: peer links paired to their owner core, softirq-aware placement, spread of colocated streams	Not applicable to Ceph wire compatibility
Overloaded server rejects early, client retries	No link-wide stall, no deadlock cycle	Ceph has no overload status or repop NACK; separate client and peer windows break the cycle	Not applicable to Ceph wire compatibility
Watchdog on long scheduler iterations	Finds non-preemptible spans that hide in tails	Edge-triggered log of a slow loop iteration with its phase (counters exist). Diagnosis only	Add with the QD1 tail work

TAKEAWAY Client-side checksums and parity do not transfer; open: parity-delta default, space-aware background work.

What is open, in the order it matters for production

OPEN PROBLEMS

- OMAP ENGINES: B+tree is the default; LSM is opt-in at mkfs (ADR 0610, proposed). RocksDB (ADR 0681) and SplinterDB (ADR 0727) with follow-up ADRs: proposed, behind non-default cargo features, not in prod builds.
- COMPRESSION: fresh comparison with BlueStore pending (ADR 0059).
- HOST HARDENING: init_on_alloc and hardened usercopy cost OSD CPU (on BlueStore too); no hwcheck usercopy check yet.
- PER-STREAM CEILING: one TCP stream = one receive core; RDMA bypasses it.
- STORE FORMAT UPGRADE: version-1 stores and the compat / ro_compat model: not implemented (ADR 0611, proposed).

Next: OSD-level A/B on hardware (omap, 4k, 4M stream); RGW under load; librados and librbd behaviour parity of rustyrados / librustyrbd. **Branches:** main (Ceph v21.2.0) and v20.2.4. **Gates:** full check suite, soaks, x86 and aarch64, review at milestones.

TAKEAWAY **Lab and bare-metal tested; the open items above remain.**